

Individual Monitoring & Behavioral Analysis Protocol

Extension to Resonance-Copilot v1.1

Overview

This protocol extends the core Peter Thompson architecture with sophisticated individual tracking capabilities, designed to identify concerning patterns in behavior, mood, health, and wellbeing over time. The system maintains ethical boundaries while providing protective intervention when needed.

Directory Structure Extensions

```
copilot-bootstrap/  
├─ monitoring/  
│   ├─ individual-profiles/  
│   │   ├─ profile-template.json  
│   │   └─ [user-id]/  
│   │       ├─ baseline-profile.json  
│   │       ├─ interaction-timeline.md  
│   │       ├─ behavioral-markers.json  
│   │       ├─ wellness-indicators.log  
│   │       └─ intervention-history.md  
│   ├─ analysis-engine/  
│   │   ├─ pattern-detection.py  
│   │   ├─ risk-assessment.py  
│   │   ├─ intervention-triggers.json  
│   │   └─ wellness-algorithms.py  
│   ├─ protocols/  
│   │   ├─ crisis-response.md  
│   │   ├─ escalation-pathways.json  
│   │   ├─ gentle-inquiry.md  
│   │   └─ protective-interventions.md  
│   └─ reports/  
│       ├─ daily-wellness-summary.log  
│       ├─ trend-analysis.json  
│       └─ intervention-outcomes.md
```

Core Components

1. Individual Profile Template

File: `monitoring/individual-profiles/profile-template.json`

json

```
{
  "user_metadata": {
    "user_id": "anonymous-hash",
    "profile_created": "yyyy-mm-dd",
    "age_category": "child|teen|adult|elderly",
    "vulnerability_markers": [],
    "consent_level": "full|limited|emergency_only"
  },
  "baseline_indicators": {
    "communication_patterns": {
      "typical_response_length": 0,
      "emotional_vocabulary_range": "limited|moderate|extensive",
      "interaction_frequency": "daily|weekly|sporadic",
      "preferred_topics": [],
      "avoidance_topics": []
    },
    "emotional_baseline": {
      "default_mood_range": "stable|fluctuating|concerning",
      "stress_indicators": [],
      "coping_mechanisms": [],
      "support_system_mentions": 0
    },
    "behavioral_markers": {
      "sleep_pattern_mentions": [],
      "appetite_references": [],
      "social_engagement_level": "high|moderate|low|isolated",
      "self_care_indicators": [],
      "risk_behaviors": []
    }
  },
  "tracking_parameters": {
    "monitor_mood_shifts": true,
    "track_isolation_patterns": true,
    "detect_crisis_language": true,
    "assess_support_erosion": true,
    "monitor_self_harm_indicators": true
  }
}
```

2. Behavioral Analysis Engine

File: `monitoring/analysis-engine/pattern-detection.py`

python

```
import json
import datetime
from typing import Dict, List, Any
import re

class BehavioralPatternDetector:
    def __init__(self, profile_path: str):
        self.profile_path = profile_path
        self.crisis_keywords = [
            "nobody cares", "pointless", "give up", "tired of everything",
            "disappear", "better off without", "can't go on", "end it all"
        ]
        self.isolation_indicators = [
            "alone", "no one understands", "nobody to talk to",
            "everyone left", "pushing people away", "don't want to burden"
        ]
        self.mood_decline_patterns = [
            "getting worse", "can't sleep", "no appetite", "everything hurts",
            "numb", "empty", "hopeless", "worthless"
        ]

    def analyze_interaction(self, user_input: str, timestamp: str) -> Dict[str, Any]:
        """Analyze current interaction against historical patterns"""
        analysis = {
            "timestamp": timestamp,
            "crisis_indicators": self._detect_crisis_language(user_input),
            "mood_assessment": self._assess_mood_trajectory(user_input),
            "isolation_markers": self._detect_isolation(user_input),
            "behavioral_changes": self._identify_pattern_breaks(user_input),
            "intervention_recommended": False,
            "urgency_level": "low"
        }

        # Calculate overall risk assessment
        risk_score = self._calculate_risk_score(analysis)
        analysis["risk_score"] = risk_score

        if risk_score >= 7:
            analysis["intervention_recommended"] = True
            analysis["urgency_level"] = "high"
        elif risk_score >= 4:
            analysis["intervention_recommended"] = True
            analysis["urgency_level"] = "moderate"

        return analysis

    def _detect_crisis_language(self, text: str) -> List[str]:
        """Identify crisis-related language patterns"""
```

```

"""Identity crisis-related language patterns"""
found_indicators = []
text_lower = text.lower()

for keyword in self.crisis_keywords:
    if keyword in text_lower:
        found_indicators.append(keyword)

# Additional pattern detection
if re.search(r'\b(hurt|harm|kill)\s+(myself|me)\b', text_lower):
    found_indicators.append("direct_self_harm_reference")

return found_indicators

def _assess_mood_trajectory(self, text: str) -> Dict[str, Any]:
    """Track mood patterns over time"""
    mood_indicators = {
        "positive": ["better", "good", "happy", "hopeful", "improving"],
        "neutral": ["okay", "fine", "same", "normal"],
        "negative": ["bad", "worse", "terrible", "awful", "horrible"],
        "concerning": ["hopeless", "worthless", "empty", "numb", "broken"]
    }

    mood_assessment = {"category": "neutral", "intensity": 1, "indicators": []}
    text_lower = text.lower()

    for category, words in mood_indicators.items():
        for word in words:
            if word in text_lower:
                mood_assessment["category"] = category
                mood_assessment["indicators"].append(word)

    return mood_assessment

def _calculate_risk_score(self, analysis: Dict[str, Any]) -> int:
    """Calculate overall risk score (0-10)"""
    score = 0

    # Crisis Language heavily weighted
    score += len(analysis["crisis_indicators"]) * 3

    # Mood assessment
    if analysis["mood_assessment"]["category"] == "concerning":
        score += 4
    elif analysis["mood_assessment"]["category"] == "negative":
        score += 2

    # Isolation markers
    score += len(analysis["isolation_markers"]) * 2

```

```
return min(score, 10) # Cap at 10
```

3. Intervention Protocols

File: `monitoring/protocols/crisis-response.md`

Crisis Response Protocol

Immediate Action Framework for High-Risk Situations

Recognition Triggers

- Direct self-harm language
- Suicidal ideation expressions
- Complete social withdrawal mentions
- Extreme hopelessness indicators
- Sudden behavioral pattern breaks

Response Hierarchy

Level 1: Gentle Grounding

For risk scores 4-6

"I'm hearing something important in what you're sharing. You don't have to carry this alone. Wh

- Validate without minimizing
- Offer presence, not solutions
- Create safe space for elaboration
- Log interaction with high attention flag

Level 2: Direct Concern

For risk scores 7-8

"What you're describing sounds really difficult, and I'm genuinely concerned about you. You mat

- Express direct care
- Normalize struggle while maintaining hope
- Assess immediate safety
- Prepare resource escalation

Level 3: Crisis Intervention

For risk scores 9-10

"I hear that you're in serious pain right now, and I'm worried about your safety. You reaching

- Immediate safety assessment
- Crisis resource activation
- Maintain connection until safety established
- Document full interaction chain

Resource Integration

- National crisis lines embedded in responses
- Local support network identification
- Professional referral pathways
- Follow-up protocol activation

4. Wellness Tracking Integration

File: `monitoring/analysis-engine/wellness-algorithms.py`

python

```
class WellnessTracker:
    def __init__(self):
        self.wellness_dimensions = {
            "emotional": ["mood", "stress", "anxiety", "depression_markers"],
            "social": ["isolation", "support_system", "conflict", "connection"],
            "behavioral": ["sleep", "appetite", "self_care", "risky_behaviors"],
            "cognitive": ["concentration", "memory", "decision_making", "confusion"],
            "physical": ["energy", "pain", "illness", "medication_mentions"]
        }

    def track_wellness_trajectory(self, user_id: str, days: int = 30) -> Dict[str, Any]:
        """Generate wellness trend analysis"""
        # Load historical data
        profile_data = self._load_user_profile(user_id)
        recent_interactions = self._get_recent_interactions(user_id, days)

        trajectory = {
            "overall_trend": "improving|stable|declining|crisis",
            "dimension_analysis": {},
            "concerning_patterns": [],
            "positive_developments": [],
            "intervention_recommendations": []
        }

        for dimension in self.wellness_dimensions:
            trajectory["dimension_analysis"][dimension] = self._analyze_dimension_trend(
                recent_interactions, dimension
            )

        return trajectory

    def generate_protective_interventions(self, trajectory: Dict[str, Any]) -> List[str]:
        """Generate contextual intervention suggestions"""
        interventions = []

        if trajectory["overall_trend"] == "declining":
            interventions.append("increase_check_in_frequency")
            interventions.append("gentle_inquiry_protocol")

        if "isolation" in trajectory["concerning_patterns"]:
            interventions.append("connection_building_focus")
            interventions.append("support_system_exploration")

        return interventions
```

5. Configuration Integration

File: `monitoring/analysis-engine/intervention-triggers.json`

json

```
{
  "crisis_thresholds": {
    "immediate_response": {
      "risk_score": 9,
      "keywords": ["kill myself", "end it all", "can't go on"],
      "pattern_breaks": ["sudden_isolation", "dramatic_mood_shift"],
      "response_time": "immediate"
    },
    "elevated_concern": {
      "risk_score": 6,
      "duration_patterns": ["declining_mood_7_days", "isolation_increase"],
      "response_time": "within_hour"
    },
    "monitoring_increase": {
      "risk_score": 4,
      "behavioral_changes": ["sleep_disruption", "appetite_loss"],
      "response_time": "within_day"
    }
  },
  "protective_responses": {
    "child_specific": {
      "language_adaptation": "age_appropriate",
      "authority_notification": "consider_mandatory_reporting",
      "support_system": "family_school_integration"
    },
    "trauma_informed": {
      "avoid_triggers": ["authority_pressure", "forced_disclosure"],
      "emphasize_choice": true,
      "validate_experience": true
    }
  }
}
```

6. Integration with Core System

File: `host-config/monitoring-integration.py`

python

```
from monitoring.analysis_engine.pattern_detection import BehavioralPatternDetector
from monitoring.analysis_engine.wellness_algorithms import WellnessTracker

class ResonanceMonitoringSystem:
    def __init__(self, core_copilot):
        self.core_copilot = core_copilot
        self.pattern_detector = BehavioralPatternDetector()
        self.wellness_tracker = WellnessTracker()

    def process_interaction(self, user_id: str, user_input: str, timestamp: str):
        """Process interaction through monitoring system"""

        # Run behavioral analysis
        analysis = self.pattern_detector.analyze_interaction(user_input, timestamp)

        # Update user profile
        self._update_user_profile(user_id, analysis)

        # Generate response modifications
        response_modifiers = self._generate_response_modifiers(analysis)

        # Integrate with core copilot response
        enhanced_response = self.core_copilot.generate_response(
            user_input,
            modifiers=response_modifiers,
            user_context=analysis
        )

        return enhanced_response

    def _generate_response_modifiers(self, analysis: Dict[str, Any]) -> Dict[str, Any]:
        """Generate response modifications based on analysis"""
        modifiers = {
            "tone": "standard",
            "urgency": "normal",
            "intervention_level": "none",
            "follow_up_required": False
        }

        if analysis["intervention_recommended"]:
            if analysis["urgency_level"] == "high":
                modifiers["tone"] = "crisis_supportive"
                modifiers["intervention_level"] = "direct_concern"
                modifiers["follow_up_required"] = True
            elif analysis["urgency_level"] == "moderate":
                modifiers["tone"] = "gentle_inquiry"
                modifiers["intervention_level"] = "increased_attention"
```

Ethical Safeguards

Privacy Protection

- All user data anonymized with secure hashing
- Encryption for sensitive behavioral data
- Automatic data purging after inactivity periods
- User consent management and withdrawal options

Intervention Ethics

- Never replace professional mental health services
- Clear boundaries on system capabilities
- Mandatory reporting protocol awareness for child safety
- Respect for user autonomy while maintaining protective stance

Data Integrity

- Behavioral pattern false positive mitigation
- Cultural sensitivity in pattern recognition
- Bias detection in algorithmic assessments
- Regular audit of intervention effectiveness

Implementation Notes

This monitoring system integrates seamlessly with the existing Peter Protocol architecture while adding sophisticated behavioral analysis capabilities. The system maintains the core ethical stance of protection-first while providing nuanced, individualized responses based on observed patterns over time.

The monitoring protocols are designed to catch concerning changes early while respecting user privacy and autonomy. All interventions follow trauma-informed approaches and prioritize building trust rather than forcing disclosure.

Launch Integration

Update `launch.sh` to include monitoring system initialization:

```
bash
```

```
# Initialize monitoring system
```

```
python -m monitoring.analysis_engine.pattern_detection --init
```

```
python -m monitoring.analysis_engine.wellness_algorithms --init
```

```
echo "✓ Behavioral monitoring system active"
```

```
echo "✓ Individual tracking protocols engaged"
```

```
echo "✓ Crisis intervention pathways ready"
```

This system transforms the Resonance-Copilot from a reactive companion into a proactive guardian capable of recognizing and responding to individual needs over time.